CONCEPT

Day 2 | connect the components, then operate them

Today's core

- Service Bus and real telemetry as individual services.
- A realistic document-processing scenario.
- Evidence of success, failure and recovery.

After the core

- CI/CD and cost management essentials.
- Optional Bicep / ARM and CLI / PowerShell.
- Know what is implemented versus target architecture.

Prepared-demo deck: new Azure integration and automation are not yet implemented.

REFERENCE / Day 2 route and integration readiness

Student route

CONCEPT

Priorities: core first, optional detail last

Protected core

- Remaining services: 60-75 min.
- Whole-workflow demo + discussion: about 90 min.
- Failure/recovery verification and attendee questions.

Delivery / operations

- CI/CD + cost essentials: about 45 min.
- IaC / CLI / PowerShell extras: 30-45 min if time.
- Use remaining time for practice, Q&A and cleanup.

CONCEPT

Why put a queue between upload and processing?

Synchronous request

- Browser waits for every downstream operation.
- A slow dependency lengthens the response.
- Retries can duplicate work without safeguards.

Asynchronous workflow

- Accept, persist and acknowledge the upload.
- Worker processes independently.
- Status tells the user what happened.
- Delivery, retries and duplicates need explicit design.

CONCEPT

Queue vocabulary you must distinguish

Message lifecycle

- Send: put a work notification on the queue.
- Peek: inspect without settling the message.
- Receive with lock: attempt processing.
- Complete: acknowledge successful handling.

Failure handling

- Abandon / lock expiry allows redelivery.
- Duplicate delivery requires idempotent processing.
- Dead-letter is for delivery/processing problems.
- Needs review is a business outcome, not automatically a dead letter.

REFERENCE / Microsoft reference; Basic tier supports queues, not topics

Microsoft: Service Bus Explorer

Microsoft: Service Bus .NET

Service Bus | small messages, right tier, bounded practice

What drives cost?

- Tier and messaging operations affect billing.
- Standard and Premium have capacity / baseline elements.
- Premium capacity can cost even when the queue is idle.

Student cost-saving habits

- Use the assigned queue and a few fictional messages.
- Send references, not document bodies.
- Avoid endless receive loops and unnecessary repeated peeks.
- Do not create namespaces or upgrade tiers without approval.

Use approved existing resources. Ask before changing tiers, retention, capacity or shared settings.

Official service pricing / guidance (check current region and offer)

DEMO

Demo 09 | portal queue -> VS Code client

Portal | prepared queue

- Confirm assigned namespace, queue and data role.
- Send a fictional reference-only test message.
- Peek and inspect the message without completing it.

VS Code | prepared .NET client

- Use Entra identity; no connection-string shortcut.
- Receive with a lock; print safe reference/message ID.
- Complete only after successful handling.
- Check portal queue state after the client exits.

PREPARATION REQUIRED | Do not improvise writes to a shared queue.

REFERENCE / Supplied portal lab; a .NET receive/complete client still needs preparation

Service Bus portal lab

Microsoft: Service Bus .NET

EXERCISE

Exercise 06 | trace one message

Do only after rehearsal | 15 min

- Use your assigned queue and fictional reference.
- Send -> peek -> receive -> complete.
- Record client settlement and portal observation.

Discuss

- What happens if the receiver exits before completion?
- Why can the same message be processed again?
- How would you avoid two status updates?
- No prepared client/access? Use a discussion, not a pretend pass.

REFERENCE / Supplied portal lab; prepare .NET client and scoped access before this extension

Service Bus portal lab

Microsoft: Service Bus .NET

CONCEPT

Application Insights + Log Analytics

Application Insights

- Application requests, dependencies and failures.
- Operations are linked with correlation identifiers.
- Instrumentation must actually be configured.

Log Analytics

- Workspace stores and queries relevant tables.
- Confirm resource scope before choosing schema.
- AppRequests in workspace; requests may appear in AI scope.
- Day 1 synthetic KQL is not proof of ingestion.

REFERENCE / Logs guide section 6; optional real telemetry

Logs / KQL handout

Microsoft: Application Insights

Application Insights | useful telemetry, not unlimited logging

What drives cost?

- Telemetry volume and the workspace retention policy.
- Extra tests, alerts and data exports may add charges.
- Duplicate instrumentation can duplicate telemetry volume.

Student cost-saving habits

- Generate a few known requests, not a load test.
- Use scoped queries and short time windows.
- Avoid document bodies, secrets and noisy debug logs.
- Owner reviews sampling and caps; caps can hide incidents.

Use approved existing resources. Ask before changing tiers, retention, capacity or shared settings.

Official service pricing / guidance (check current region and offer)

DEMO

Demo 10 | invoke, then find the request

Generate a known event

- Use the approved Function endpoint.
- Record UTC request time; keep key private.
- Confirm the response before searching telemetry.

Find it in the portal

- Open the linked workspace; inspect Tables.
- If AppRequests exists, use a short time filter.
- Confirm actual AppRoleName; inspect matching rows.
- Allow ingestion delay; zero rows != missing table.

A 200 response does not itself prove a request was collected in this workspace.

REFERENCE / Logs guide section 6; instrumentation/real data still needs rehearsal

Logs / KQL handout

App Insights portal lab

EXERCISE

Exercise 07 | useful operators on real data

Workspace query | confirm schema

```
AppRequests
| where TimeGenerated > ago(1h)
| where AppRoleName =~
"fristAzureFunctionApp"
| summarize Requests=count() by ResultCode
| order by Requests desc
```

Questions and checks

- Run only after confirming the role/table.
- Which result codes appear?
- Use project / take to inspect a small sample.
- Missing data? Record it; do not invent a chart.

REFERENCE / Section 6 plus Microsoft KQL tutorial; counts will vary

Logs / KQL handout

Microsoft: KQL tutorial

CONCEPT

A fictional document check, not a lending decision

User journey

- Submit a fictional application reference + text file.
- See processing status and a useful outcome.
- Valid format -> Completed.
- Missing/mismatched fields -> Needs review.

Safety boundary

- No customer, personal or financial data.
- Training marker is not a privacy detector.
- Current tracker accepts small UTF-8 .txt files.
- No credit scoring or identity verification.

REFERENCE / Real-world training scenario and SampleData

Tracker setup guide

Tracker README

Fictional test cases

DEMO

Demo 11 | the working local workflow

Start locally

- Open Labs\LbsDocumentTracker.
- Use README Development-only startup commands.
- Open localhost:5193; explain in-memory state.
- Submit sample 1001, then sample 1002.

Prove the outcomes

- 1001: Completed.
- 1002: Needs review; explain missing document type.
- Inspect dashboard and detail page.
- Restart loses data: persistence is not implemented.

IMPLEMENTED LOCAL DEMO | No Azure clients, no authentication; never host as-is.

REFERENCE / README Run locally; SampleData/cases.json

Tracker setup guide

Tracker README

Fictional test cases

EXERCISE

Exercise 08 | explain both validation outcomes

Do | 15-20 min

- Run supplied local scenario checks or use the UI.
- Try one valid and one invalid fictional case.
- Use the exact file/reference pairing.
- Capture status only, not document contents.

Checkpoint

- Explain format rejection versus business Needs review.
- Name what disappears on restart.
- Identify the current in-memory queue/store.
- Explain why this starter cannot be cloud-hosted unchanged.

REFERENCE / Run scenario checks; expected outcomes

Tracker README

Fictional test cases

CONCEPT

From local memory to an Azure workflow

Target data path | NOT implemented

- Authenticated website accepts the upload.
- Private Blob holds file; SQL holds state.
- Durable dispatch queues a reference.
- Function validates and updates status.

Target operational path

- Scoped managed identities reach dependencies.
- Correlation links request, message and worker.
- Retries are safe and observability is bounded.
- Approved hosting is separate from the FC1 Function plan.

TARGET DESIGN | Resource existence is not application integration.

REFERENCE / Solid local path vs dashed Azure target

Local / target architecture

Tracker README

CONCEPT

The dangerous gap: stored, but not queued

Failure sequence

- Upload is stored successfully.
- SQL status is written.
- Queue send fails or process crashes.
- Without recovery, work remains stranded.

Design response

- Use a durable dispatch / outbox strategy.
- Track dispatch state; retry safely.
- Worker uses a stable idempotency key.
- Complete the message only after durable outcome.

REFERENCE / Next Azure milestone; durable dispatch requirement

Tracker README

CONCEPT

The application is not the instructor's account

Separate identities

- Attendee: exercises and approved deployment.
- Web app: scoped SQL/Blob/queue operations.
- Worker: scoped receive/read/update operations.
- Pipeline: only approved deployment scope.

Do not carry shortcuts forward

- Function key is not end-user sign-in.
- No secrets embedded in code or templates.
- Network reachability still applies to identities.
- No database-wide or subscription-wide grant by default.

REFERENCE / Identity design before hosting

Microsoft: managed identities

Tracker README

Identity and networking | secure does not mean cost-free

What drives cost?

- Basic role assignment is not a reason to buy more capacity.
- Premium identity features can require licences.
- Private endpoints, gateways, firewalls and traffic may incur charges.

Student cost-saving habits

- Use existing approved identities and network paths.
- Do not disable security to save money.
- Do not deploy a new gateway / firewall for a small lab.
- Have the owner review security, dependencies and pricing together.

Use approved existing resources. Ask before changing tiers, retention, capacity or shared settings.

Official service pricing / guidance (check current region and offer)

Security | check the network, identity and permissions

Security: independent checks, not one magic permission

1. Network

Can the request reach it?

Firewall / private endpoint
Approved VPN / DNS route

2. Identity

Who is calling?

User sign-in / managed identity
Authenticate with approved method

3. Permission

May this identity do this?

Scoped Azure data role
SQL user + table permission

4. Protection

Is data handled safely?

TLS / private data / safe logs
Never share secrets or keyed URLs

Least privilege: scope the role to the smallest approved target

Subscription > resource group > resource > supported child scope (for example one Blob container)

Azure RBAC assignments normally inherit downward; deny assignments, conditions and service rules still apply.

Azure Reader != Blob data access != SQL table permission. A network allow-list grants no data permission.

Function key: a shared endpoint secret, NOT a person's sign-in. Regenerate exposed keys with the owner.

A 403 alone does not identify the failed check. Diagnose the exact error; do not broaden access blindly.

DEMO

Demo 12 | whole workflow, one reference

Only with the prepared Azure app

- Show architecture and target identities first.
- Submit one approved fictional file.
- Observe Blob + SQL, then message + worker.
- Refresh user status; find correlated telemetry.

Evidence to show side by side

- Portal: exact container/table/queue evidence.
- VS Code: relevant code and safe logs.
- One reference connects the evidence chain.
- Compare expected and actual states, not just HTTP 200.

NOT LIVE-READY | If unprepared, use Demo 11 + architecture discussion instead.

REFERENCE / Target-design references only; Azure integration runbook not yet available

Tracker README

Local / target architecture

Fictional test cases

EXERCISE

Exercise 09 | acceptance checklist

Success case

- Authorised user can submit fictional data.
- File and state persist in assigned stores.
- Work reaches a durable outcome.
- UI and data evidence agree.

Recovery / protection case

- No duplicate processing outcome on redelivery.
- Queue/send failure has a recovery route.
- Other users' files/status cannot be accessed.
- No keys, raw document text or sensitive IDs in logs.

REFERENCE / Design exercise until integrated implementation exists

Tracker README

Local / target architecture

CONCEPT

A pipeline repeats what you have already tested

Continuous integration

- Commit / pull request triggers checks.
- Restore -> build -> tests -> package.
- Fail visibly; keep deployable artifact traceable.

Controlled deployment

- Approve environment and exact target.
- Authenticate with federated workload identity.
- Deploy artifact -> run smoke test.
- Retain a known-good rollback strategy.

REFERENCE / Official references; no course pipeline implemented yet

Microsoft: Functions + GitHub Actions

Microsoft: GitHub OIDC to Azure

DEMO

Demo 13 | a safe pipeline walkthrough

Prepared repository only

- Point to trigger, jobs, tests and artifact.
- Explain OIDC federation and scoped Azure role.
- Show environment approval before deployment.
- Inspect run logs without exposing credentials.

Prove the result

- Commit/run -> artifact -> deployment identity.
- Smoke test confirms expected app behaviour.
- Show how a failed test blocks release.
- Explain rollback; do not assume every plan supports slots.

PREPARATION REQUIRED | No live push/deployment is authorised by the deck.

REFERENCE / Microsoft walkthroughs; pipeline demo requires separate preparation

Microsoft: Functions + GitHub Actions

Microsoft: GitHub OIDC to Azure

CONCEPT

Cost visibility is not an automatic spending cap

Read the bill with context

- Scope: approved subscription / resource group.
- Filter by service, date and resource.
- Compare actual and forecast; data can be delayed.
- Use tags and ownership to explain spend.

Control the training footprint

- Budget alerts notify; they do not stop spend.
- Choose approved tiers before provisioning.
- Stopping a host may not remove all charges.
- Cleanup has an owner; shared resources are not disposable.

REFERENCE / Official Cost Analysis and budgets guidance

Microsoft: Cost Analysis

Microsoft: budgets

Course-wide cost checklist | alerts are not a hard cap

What drives cost?

- Idle compute and supporting resources can still charge.
- Pricing varies by region, tier, currency and agreement.
- Budgets notify; they do not automatically stop spending.

Student cost-saving habits

- Check scope and date filters in Cost Analysis.
- Tag ownership where permitted and record what you create.
- Agree a cleanup owner and date before creating anything.
- No resource deletion or budget changes without approval.

Use approved existing resources. Ask before changing tiers, retention, capacity or shared settings.

Official service pricing / guidance (check current region and offer)

DEMO

Demo 14 | inspect cost without changing policy

Portal

- Open Cost Management at authorised scope.
- Select a date range and resource-group filter.
- Group by service/resource; compare actual vs forecast.
- Review budget concept; create only if approved.

Attendee discussion

- Which resources continue charging when idle?
- What belongs to this lab versus shared hosting?
- What can be deleted safely, by whom?
- No cost access? Explain with a labelled example, not invented live figures.

REFERENCE / Official references; budget creation is optional and requires approval

Microsoft: Cost Analysis

Microsoft: budgets

CONCEPT

Bicep and ARM: two views of Azure deployment

Bicep

- Declarative resource definitions.
- Parameters express environment differences.
- Build compiles Bicep into ARM JSON.
- No application binary is produced by Bicep build.

ARM deployment

- Azure Resource Manager applies resource changes.
- Validate and what-if before approved deployment.
- Review role, scope, region, SKU and cost.
- A what-if is a preview, not a guarantee of harmlessness.

REFERENCE / Optional topic; no new course IaC template included

Microsoft: Bicep

Microsoft: what-if

DEMO

Demo 15 | inspect before you deploy

Prepared template walkthrough

- Open a reviewed training .bicep file.
- Explain parameters and generated ARM JSON.
- Build/lint; validate at the intended scope.
- Review what-if additions, changes and deletions.

Decision checkpoint

- Correct subscription/group/region?
- No embedded credentials or broad network access?
- Expected cost and role assignments approved?
- Only then deploy; otherwise stop at preview.

OPTIONAL | Do not invent infrastructure live or treat a compiled file as a deployment.

REFERENCE / Official guidance; prepared template required

Microsoft: Bicep

Microsoft: what-if

CONCEPT

PowerShell terminal != Azure PowerShell module

Azure CLI | az

- Commands can run inside PowerShell.
- Own sign-in/session: az login.
- JSON/TSV output; check $LASTEXITCODE.
- Already used in Blob and KQL labs.

Azure PowerShell | Az module

- Cmdlets such as Connect-AzAccount.
- Own context; inspect before running commands.
- Returns PowerShell objects.
- Install only through approved tooling process.

REFERENCE / Official getting-started references

Microsoft: Azure CLI

Microsoft: Azure PowerShell

DEMO

Demo 16 | same read-only lookup, two tools

Azure CLI

```
az account show --query name -o tsv
az group show --name "Training"
Check subscription before resource lookup.
No create / delete commands in this demo.
```

Azure PowerShell

- Get-AzContext
- Get-AzResourceGroup -Name "Training"
- Requires approved Az module and sign-in.
- Compare the returned group/location.

REFERENCE / Official references; replace group with your assigned name

Microsoft: Azure CLI

Microsoft: Azure PowerShell

CONCEPT

Tips that prevent the most classroom failures

Before commands

- Open the correct folder and a fresh terminal.
- Quote paths; replace placeholders.
- Check context and current IP.
- Keep a read-only check ready.

Before declaring success

- Check native exit codes and expected output.
- Distinguish no rows from a missing table.
- Separate local, cloud, UI and automation evidence.
- Record Blocked honestly and keep a fallback.

REFERENCE / Component troubleshooting; evidence boundaries

Student route

EXERCISE

Finish with a plan, not leftover resources

Attendee exit ticket

- Trace one reference across the target workflow.
- Explain idempotency and durable dispatch.
- Name one pipeline and one cost safeguard.
- Identify what remains to be implemented.

Instructor cleanup

- Stop only your own local processes.
- Remove temporary data/rules only with approval.
- Do not delete shared hosting, SQL or workspaces.
- Keep evidence and assign follow-up owners.

REFERENCE / Cleanup guidance and readiness gate

Student route

Tracker README

CONCEPT

Reference map | prepared versus planned

Existing local documents

- Start Here and individual component handouts.
- Tracker setup guide, README and sample cases.
- Architecture distinguishes local from target.
- Record your own exercise results and limitations.

Official references for planned demos

- Service Bus and real telemetry.
- GitHub Actions / OIDC and cost budgets.
- Bicep / what-if and CLI / Az PowerShell.
- Official tutorials are not completed course labs.

REFERENCE / Each demo/exercise slide has its own clickable references

Student route

Tracker README